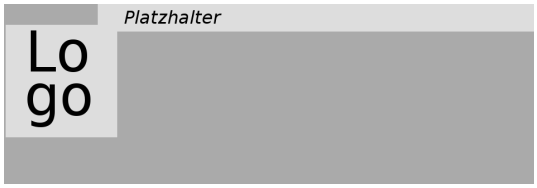


Term paper

Sample title for a report

Author:	First name Last name
Matriculation No.:	xxxxxxx
Course of Studies:	Business Intelligence and Data Analytics
First examiner:	Prof. Dr. Elmar Wings
Submission date:	April 14, 2026

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Contents

List of Figures

List of Listings

Acronyms

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

1. Introduction

Flettner rotors are now widely used [1,2,3]. ...

The construction of a Flettner rotor for a water taxi [4] ...

External influences pose great challenges to [5,6] ...

Through the use of 3D printed parts, ...

In the second chapter, ...

Part I.

Domain Knowledge - Application

2. Application

Part II.

Domain Knowledge - Hardware

3. Hardware XY

4. Lichtschranke

4.1. General

Der Begriff Sensor stammt aus dem Lateinischen und bedeutet sinngemäß „Fühler “. Sensoren dienen zur qualitativen und quantitativen Auffassung von physikalischen, chemischen, biologischen, klimatischen und medizinischen Größen. [HS23] Hierbei bestehen Sensoren immer aus einem Sensorelement, welches sich mit Veränderung der physikalischen Gegebenheiten ebenfalls in seinen Eigenschaften verändert. Besonders die Veränderung der elektrischen Eigenschaften des Sensorelementes ist hierbei von Interesse. Ein Beispiel sind Materialien, welche ihren Widerstand bei Erwärmung verändern [Rod23]. Zu jedem Sensorelement wird zudem eine Auswerteelektronik benötigt. Die Auswerteelektronik wandelt dann die beispielhafte Veränderung der elektrischen Eigenschaften in ein eindeutiges analoges oder digitales Signal. Dieses Signal kann dann an ein verarbeitendes System, beispielsweise einen Mikrocontroller, übergeben werden [HS23].

4.2. Optische Näherungssensoren

Optische Näherungssensoren gehören zu den berührungslos arbeitenden Sensoren und werden zur Erfassung von Objekten oder Abständen eingesetzt, ohne dass ein mechanischer Kontakt erforderlich ist. Um diese Eigenschaft zu ermöglichen, bestehen optische Näherungssensoren grundsätzlich aus einem Sender und einem Empfänger. Die Komponenten basieren auf den grundlegenden physikalischen Prinzipien der Elektrodynamik [Rod23]. Mithilfe von elektromagnetischer Strahlung können diese Sensoren physikalische Messgrößen aufnehmen und messen. Im Grundlegenden bestehen diese Sensoren ebenfalls aus Sensorelement und Auswerteelektronik, jedoch sind die Sensorelemente in ihrem Komplexitätsgrad als deutlich höher einzustufen. Oftmals werden Halbleitermaterialien und -komponenten für diese Sensorelemente verwendet [Rod23], [LM12]. Die in Halbleitermaterialien auftretenden physikalischen Effekte bei der Zufuhr von Energie sind entscheidend für die Funktionsweise vieler optischer Sensoren. Dabei werden insbesondere Prozesse wie die Absorption und Emission von Photonen sowie die daraus resultierende Erzeugung und Trennung von Ladungsträgern genutzt [RPF20]. Typische Empfängerelemente sind Photodioden oder Phototransistoren, die bei Lichteinfall einen messbaren Strom oder eine Spannungsänderung erzeugen. Als Strahlungsquellen werden häufig Leuchtdioden (LEDs) oder Laserdioden eingesetzt, da diese eine definierte Wellenlänge und hohe Intensität bereitstellen können. [LM12]

4.2.1. Aufbau von Lichtschranken

Der Sender dient als Strahlenquelle der elektromagnetischen Strahlung und emittiert diese, wohingegen der Empfänger die Strahlung empfängt und auswertet. Charakteristisch ist hierbei, dass Sender und Empfänger nicht zwangsläufig koaxial gegenüberliegend, sondern ebenfalls nebeneinanderliegend angeordnet sein können.
Der erste Fall ist charakteristisch für die Bauweise der „Einweg-Lichtschranke“, welche in der Industrie häufig bei hochpräzisen Anwendungen anzutreffen ist. Hierbei befinden sich Sender und Empfänger gegenüberliegend, sodass ein Lichtstrahl direkt vom Sender zum Empfänger übertragen wird. Eine Unterbrechung dieses Strahls durch ein Objekt führt unmittelbar zu einer Signaländerung am Empfänger. Im zweiten Fall wird ein

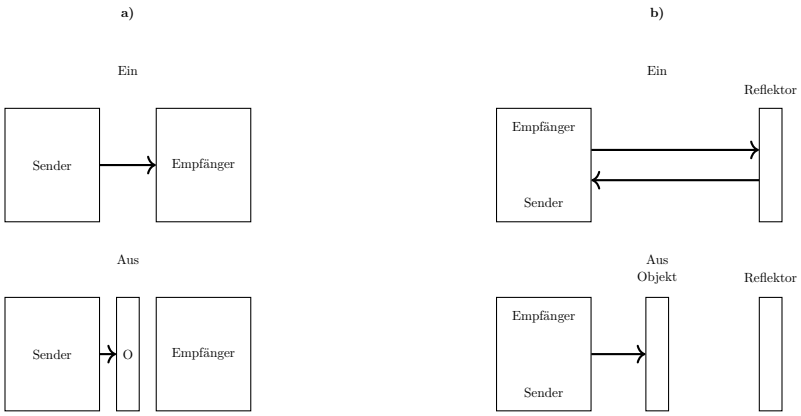


Figure 4.1.: a) Einweg-Lichtschranke, b) Reflex-Lichtschranke

Reflektor zur Rückführung des Lichtstrahls verwendet. Um dies zu ermöglichen, muss ein kleiner Winkel zwischen den optischen Achsen beider Komponenten vorhanden sein. Diese Bauart wird gängig als „Reflex-Lichtschranke“ bezeichnet [LM12]. Nachfolgende Grafik zeigt die Wirkprinzipien beider Bauarten auf 4.1.

Zusätzlich zum optischen Sender und Empfänger umfasst der Aufbau moderner Lichtschranken weitere funktionale Komponenten. Dazu gehören optische Elemente wie Linsen oder Blenden zur Strahlformung und Fokussierung, elektronische Verstärker zur Signalaufbereitung sowie Schwellwertschaltungen zur sicheren Detektion von Signaländerungen. Häufig wird das ausgesendete Licht moduliert, um Störeinflüsse durch Umgebungslicht zu minimieren. Der Empfänger ist dann auf diese Modulation abgestimmt und kann das Nutzsignal gezielt herausfiltern. [LM12]

4.2.2. Funktionsprinzip von Lichtschranken

Das Funktionsprinzip von Lichtschranken basiert auf der gezielten Auswertung von Lichtintensitäten. Der Sender erzeugt einen definierten Lichtstrahl, der sich im Raum ausbreitet. Trifft dieser Strahl auf ein Objekt, so wird er entweder unterbrochen, reflektiert oder gestreut, abhängig von der Bauart der Lichtschranke. Bei der Einweg-Lichtschranke führt eine Unterbrechung des Lichtstrahls zu einer signifikanten Reduktion der am Empfänger ankommenden Strahlungsleistung. Diese Änderung wird durch das Sensorelement detektiert und von der Auswerteelektronik in ein Schaltsignal umgesetzt. Bei der Reflex-Lichtschranke hingegen wird der Lichtstrahl zunächst von einem Reflektor zurückgeworfen. Befindet sich kein Objekt im Strahlengang, erreicht ausreichend Licht den Empfänger. Wird der Strahl durch ein Objekt unterbrochen oder abgeschwächt, sinkt die empfangene Intensität unter einen definierten Schwellenwert, wodurch ein Schaltsignal ausgelöst wird.

Das physikalische Wirkprinzip im Empfänger basiert meist auf dem inneren photoelektrischen Effekt in Halbleitern. Dabei erzeugt einfallendes Licht Elektron-Loch-Paare, was zu einer Änderung des elektrischen Stroms führt. Diese Stromänderung ist proportional zur Lichtintensität und kann somit zur Detektion von Objekten genutzt werden [RPF20]. Die Zuverlässigkeit der Messung hängt von verschiedenen Faktoren ab, darunter die Wellenlänge des verwendeten Lichts, die optischen Eigenschaften des Objekts (z. B. Reflexionsgrad), sowie Umgebungsbedingungen wie Staub, Nebel oder Fremdlicht. Durch geeignete konstruktive Maßnahmen und Signalverarbeitung können diese Einflüsse jedoch weitgehend kompensiert werden.

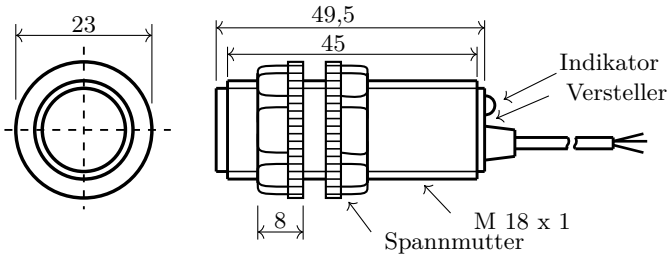


Figure 4.2.: Technische Darstellung der Produktabmaße, Alle Maße in [mm]

4.3. Reflex-Lichtschranke E18-D80N

Für den zuverlässigen Betrieb einer Reflex-Lichtschranke ist neben dem optischen Aufbau insbesondere deren korrekter elektrischer Anschluss von entscheidender Bedeutung. Die Integration in ein technisches System, beispielsweise eine speicherprogrammierbare Steuerung oder einen Mikrocontroller, erfordert die Berücksichtigung spezifischer elektrischer Kenngrößen sowie der verwendeten Ausgangsarten. Im Folgenden wird somit auf die elektrischen und technischen Eigenschaften sowie den korrekten Anschluss der Reflex-Lichtschranke E18-D80NK eingegangen.

4.3.1. Technische Daten der Lichtschranke

Nachfolgende Tabelle 4.1 zeigt die dem Datenblatt [RB26] entnommenen technischen Eigenschaften der Reflex-Lichtschranke. Die Grafik 4.2 zeigt die technischen Abmaße der Lichtschranke

Table 4.1.: Spezifikationen der Lichtschranke

Parameter	**Wert/Detail**
Modell	E18-D80NK
Sensormodul	nicht bekannt
Betriebsspannung	5 V DC
Stromaufnahme	ca. 100 mA
Einstellbereich	3 cm - 80 cm
Reaktionszeit	kleiner 2 ms
Erfassungswinkel	bis zu 15°
Temperaturbereich	-25°C bis +55°C
Maße	Ø 17,6 mm x 50 mm
Anschluss	Dupont Buchse

Nachfolgend wird der Aufbau und die Schaltlogik der Reflex-Lichtschranke E18-D80NK erläutert. Die Lichtschranke selbst besteht zwangsläufig aus folgenden elektrischen Komponenten [Lip18], [LM12]:

1. Anschlussleitungen für die Spannungversorgung
2. Infrarot-Leuchtdiode
3. NPN Fototransistor

4. Potentiometer
5. Kondensatoren
6. Widerstände
7. Verstärkerstufe (Operationsverstärker / Transistorverstärker)
8. Ausgangsleitung für das Signal

Als Sender dient eine Infrarot-LED, die elektromagnetische Strahlung im nicht sichtbaren Bereich erzeugt. Diese wird von einem Objekt reflektiert oder im Strahlengang beeinflusst. Der Empfänger besteht aus einer Photodiode oder einem Phototransistor. Verbaut ist ein Modul ähnlich dem Modul vom Typ TCRT5000 IR [all24]. Jedoch gibt es keinen analogen Ausgang und die Signalverstärkung ist deutlich erhöht. Ein Filter reduziert Störungen durch Fremdlicht. Ein Komparator vergleicht das Signal mit einem eingestellten Schwellwert. Dieser wird über ein Potentiometer eingestellt und bestimmt die Schaltschwelle. Der Ausgang ist als NPN-Open-Collector-Schaltung ausgeführt und liefert ein digitales Schaltsignal gegen Masse. So wird dauerhaft ein „HIGH “ auf der Signalleitung ausgegeben, solange der Sensor nicht auslöst und auf die Photodiote ein Lichtstrahl trifft. Eine interne Spannungsaufbereitung versorgt die Schaltung mit stabiler Betriebsspannung. Linsen im optischen System bündeln den Lichtstrahl. um die Reichweite zu Erhöhen und die Empfindlichkeit gegenüber Fremdlicht zu reduzieren [RB26]. Nachfolgende Grafik 4.3 zeigt einen beispielhaften Schaltplan für eine Reflex Lichtschranke mit Infrarotlicht am Beispiel des Moduls TCRT5000 IR [Lip18].

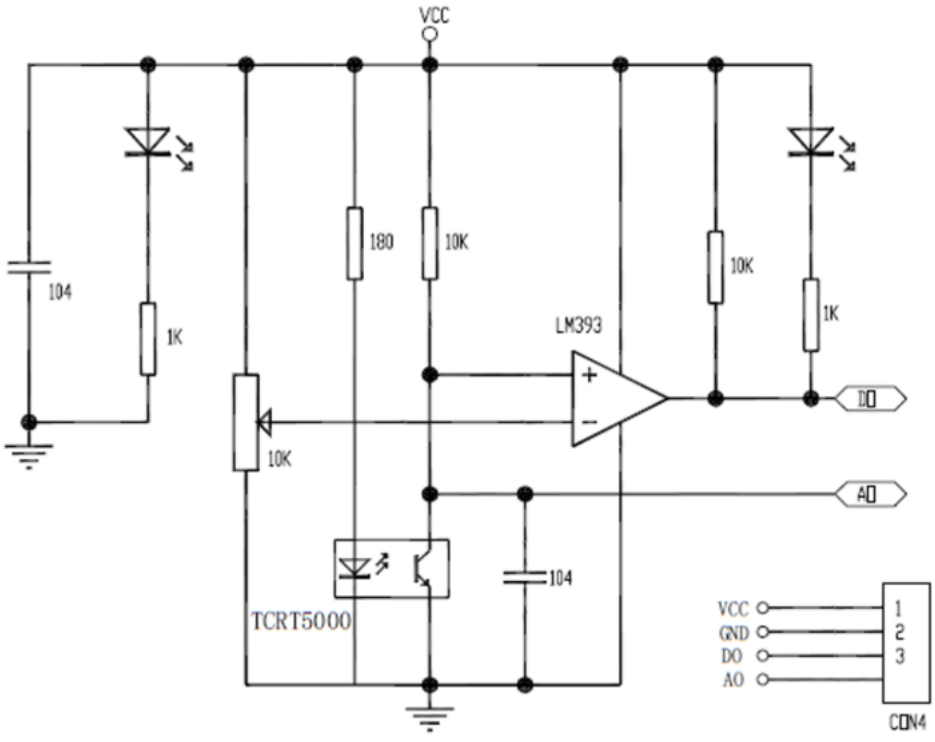


Figure 4.3.: Beispielhafte Darstellung einer Infrarot-Lichtschranke anhand des Moduls TCRT5000 IR [Lip18]

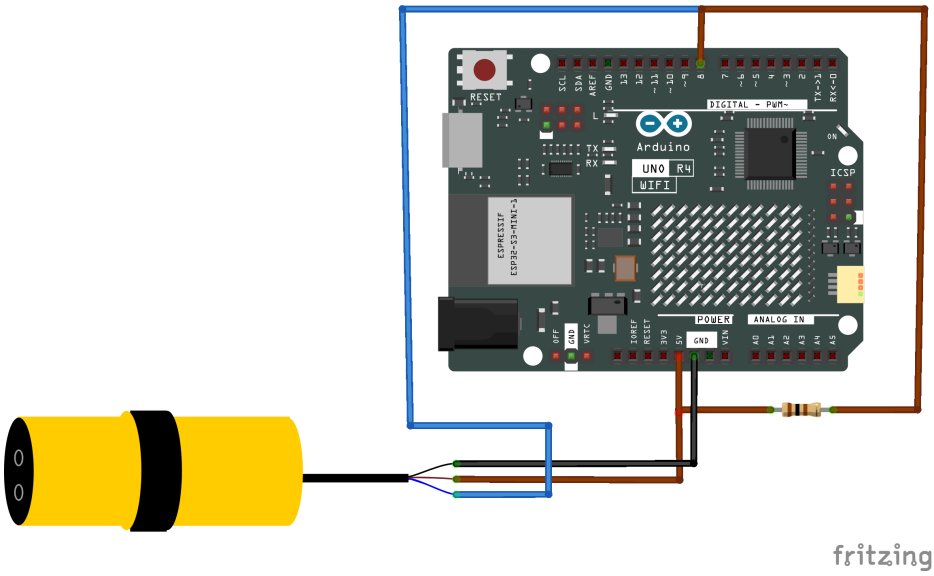


Figure 4.4.: Anschluss der Lichtschranke an einen Mikocontroller mit einem Pull-up-Widerstand ($R = 10K\Omega$)

4.3.2. Handbuch und Inbetriebnahme des Sensorsl

Folgende Vorgehensweise kann zum Anschluss und Inbetriebnahme der Lichtschranke verwendet werden.

4.3.3. Inside the Example

Um die Lichtschranke ordnungsgemäß an einen Mikrokontroller anzuschließen kann folgende Grafik 4.4 als Referenz verwendet werden. Grundsätzlich kann die benötigte Versorgungsspannung von 5V DC von einem Mikrocontoller wie beispielsweise dem Arduino UNO R4 geliefert werden. Der Ausgangspegel der Lichtschranke wird zudem die 5V ebenfalls nicht überschreiten, weshalb die Signalleitung der Lichtschranke direkt an die GPIO Pins des Arduinos geschaltet werden kann. Zudem sollten sowohl der Mikrokontroller als auch die Lichtschranke dasselbe Referenzpotenzial nutzen, um eine eindeutige Interpretation des Signals zu ermöglichen. Besonders aufgrund der Tatsache, dass die Lichtschranke gegen Masse schaltet, ist eine Verwendung eines Pull-up-Widerstands sinnvoll [Bar23]. Die Lichtschranke verwendet eine Dupont-Buchse und kann so direkt mittels Jumper-Wires mit dem Arduino verbunden werden [RB26].

4.3.4. Code

Um die Signale der Lichtschranke verarbeiten und intepretieren zu können muss das Signal durch den Mikrocontroller eingelesen und anschließend ausgewertet werden. Der Code 4.5 zeigt dieses auf. Es wird ein Eingangssignal und seine Schnittstelle definiert um anschließend ausgewertet zu werden. Da der Sensor digitale Signale liefert, muss das Programm auch nur digitale Signale auswerten.

Figure 4.5.: Dieser Code liest den digitalen Eingang 8 des Mikrocontrollers ein und erkennt anhand des Signals der Lichtschranke, ob ein Objekt den Lichtstrahl unterbricht. Der Zustand wird anschließend über die serielle Schnittstelle ausgegeben.

```
1000 /**
     * @file lichtschranke_basic.ino
1002 * @brief Simple example for connecting and reading a light barrier (E18
       -D80NK) with an Arduino Uno R4
     *
1004 * @details
     * This program demonstrates the basic integration of a light barrier
       with
1006 * an NPN open-collector output. The sensor is connected to a digital
       input pin
     * and is read cyclically.
1008 *
     * The light barrier provides an inverted signal:
1010 * - HIGH  -> no object detected
     * - LOW   -> object detected (light beam interrupted)
1012 *
     * The internal pull-up resistor of the Arduino is used, so no external
       resistor is required.
1014 *
     * @hardware
1016 * - Arduino Uno R4
     * - Light barrier E18-D80NK
1018 *
     * @connection
1020 * - Brown (VCC)  -> 5V Arduino
     * - Blue  (GND)  -> GND Arduino
1022 * - Black (OUT)  -> Digital pin 2 Arduino
     *
1024 * @author
     * Own code
1026 *
     * @date
1028 * 2026
     */
1030
     /**
1032 * @brief Digital input pin for the light barrier
     */
1034 const int sensorPin = 8;

1036 /**
     * @brief Hardware initialization
1038 *
     * @details
1040 * The sensor pin is configured as an input with internal pull-up
       resistor.
     * Additionally, the serial interface is started for debug output.
1042 */
     void setup() {
1044   // Initialize serial communication at 9600 baud
       Serial.begin(9600);
1046
       // Set sensor pin as input with internal pull-up resistor
1048   pinMode(sensorPin, INPUT_PULLUP);

1050   // Startup message
       Serial.println("Light barrier initialized");
1052 }

1054 /**
     * @brief Main program loop
1056 *
     * @details
1058 * The state of the light barrier is continuously read.
     * Depending on the signal state, it outputs whether an object is
       detected.
1060 */
     void loop() {
```

```
1062    // Read current sensor state
        int sensorState = digitalRead(sensorPin);
1064
        // Check if an object is detected
1066    if (sensorState == LOW) {
          // Light beam interrupted -> object detected
1068      Serial.println("Object detected");
        } else {
1070      // Light beam free -> no object
          Serial.println("No object");
1072    }

1074    // Small delay to stabilize output
        delay(100);
1076  }
```

../../Code/AnschlussLS/AnschlussLS.ino

4.4. Kalibirerung und Justierung

Vor der Inbetriebnahme muss die gewünschte Abstandserkennung des Sensors eingestellt werden:

1. Sensor mit der Versorgung verbinden:
   - Braun: +5 V
   - Blau: GND
2. Sensorkopf und optische Achse senkrecht auf eine Referenzfläche (Boden oder Wand) ausrichten, vorzugsweise mithilfe eines Messmittels.
3. Gewünschten Abstand zwischen Sensor und Reflektorfläche messen und Sensor in dieser Position feststellen.
4. Potentiometer (Einstellschraube am Sensor) einstellen:
   - LED AUS (Output = 1): VR im Uhrzeigersinn drehen, bis LED EIN (Output = 0) → Schaltpunkt erreicht
   - LED EIN (Output = 0): VR gegen Uhrzeigersinn drehen, bis LED AUS (Output = 1) → Schaltpunkt erreicht
5. Funktion testen:
   - Abstand ≤ Schaltabstand → LED am Sensor EIN, Output = 0
   - Abstand > Schaltabstand → LED am Sensor AUS, Output = 1

Hinweis Der Abstand zwischen Sensor und Reflektorfläche sollte so klein wie möglich gewählt werden. Sollte der Abstand zwischen Sensor und Reflektorfläche verändert worden sein, so ist eine neue Kalibrierung des Sensors notwendig.

4.5. Anwendungsbeschreibung der Lichtschranke E19-D80NK

Im Rahmen einer Anwendung wird die Lichtschranke an einem Tischförderband eingesetzt, um das Passieren eines Objektes zu erkennen und darauf basierend einen Motor zu steuern. Ziel ist es, eine zuverlässige Objekterkennung zu gewährleisten, sodass die Lichtschranke eindeutig erkennt, wenn ein Objekt den Lichtstrahl unterbricht, ohne durch kurze Störungen oder Signalflanken mehrfach ausgelöst zu werden. Der Aufbau

besteht aus einem Tischförderband, einer über dem Band montierten Lichtschranke sowie einem Arduino zur Signalauswertung. Im Betrieb fährt das Objekt in die Lichtschranke und unterbricht den Lichtstrahl, wodurch am Arduino ein LOW-Signal erkannt wird. Solange der Lichtstrahl nicht unterbrochen ist, wird ein stabiles Logiksignal erzeugt, welches vom Arduino zur Ansteuerung eines Motortreibers verwendet wird.

4.6. Tests

Um sicherzustellen, dass ein System ordnungsgemäß funktionstüchtig ist, ist es sinnvoll alle Komponenten des Systemes zu testen. Hierzu können einzelne Komponenten für sich oder im Verbund getestet werden. Das nachfolgende Kapitel beschreibt einen einfachen Test der Funktion der Lichtschranke.

4.6.1. Einfacher Funktionstest

Auf dem Arduino soll die eingebaute LED dauerhaft leuchten, hierfür wird diese ordnungsgemäß angeschlossen. Das Vorgehen zur Prüfung der ordnungsgemäßen Funktion wird nachfolgend erläutert. Zunächst wird die ArduinoIDE gestartet. Anschließend wird die Lichtschranke an die Versorgungspins (5V, GND) des Mikrocontrollers verbunden. Die Signalleitung der Lichtschranke wird daraufhin an einen GPIO-Pin des Mikrocontrollers angeschlossen, welcher bereits mit einem Pull-Up Widerstand verbunden ist. Das Anschlussschema ist identisch mit dem Anschlussschema in Abbildung 4.4. Danach wird der Mikrocontroller per USB mit dem Rechner verbunden. In der ArduinoIDE wird nun der Sketch zur Erprobung geöffnet. Der mit der Signalleitung verbundene GPIO-Pin wird an der vorgesehenen Stelle im Sketch eingetragen. Abschließend wird der Sketch auf den Mikrocontroller gespielt, um die Funktion der Lichtschranke zu überprüfen. Sollte sich kein Objekt vor der Lichtschranke befinden muss die Ausgabe ein „HIGH“ sein. [tin26]

Vorgehen zur Funktionsprüfung der Lichtschranke

1. Überprüfen aller Kabel auf festen Sitz
2. Starten der ArduinoIDE
3. Verbinden der Lichtschranke an die Versorgungspins (5V, GND) des Mikrocontrollers
4. Verbinden der Signalleitung der Lichtschranke an einen GPIO-Pin des Mikrocontrollers
5. Verbinden des Mikrocontrollers mit dem Rechner per USB
6. Öffnen der ArduinoIDE
7. Öffnen des Sketches zur Erprobung
8. Eintragen des mit der Signalleitung verbundenen GPIO-Pins an der vorgesehenen Stelle im Sketch
9. Sketch auf den Mikrocontroller spielen
10. Überprüfen der ordnungsgemäßen Funktion der Lichtschranke anhand der eingebauten LED

Nachfolgend wird der benötigte Code zum Testen der Lichtschranke beschrieben. Dieser sollte dann in der ArduinoIDE eingefügt und auf den Mikrocontroller geladen werden. Der Code liest das Signal der Lichtschranke ein und lässt eine LED aufleuchten, sobald sich das Signal in ein „LOW“ ändert. Hierzu wird Ebenfalls der Ausgang für die LED definiert. Die verwendete LED ist die auf dem Arduino verbaute LED. Auch wird die serielle Schnittstelle und das Abfrageintervall definiert. Diese werden zudem im seriellen Monitor der Arduino IDE als Ausgaben dargestellt. Sollte sich das Signal in ein „LOW“ wird zusätzlich im seriellen Monitor ein „Triggered“ ausgegeben.

Figure 4.6.: Dieser Code liest den digitalen Eingang 8 des Mikrocontrollers ein und erkennt anhand des Signals der Lichtschranke, ob ein Objekt den Lichtstrahl unterbricht. Der Zustand wird anschließend über die serielle Schnittstelle ausgegeben.

```
1000 /**
      * @brief GPIO pin for the light barrier signal.
1002  */
     #define LICHTSCHRANKE_PIN 2
1004
     /**
1006  * @brief Built-in LED pin of the Arduino UNO R4.
      */
1008 #define LED_PIN LED_BUILTIN

1010 /**
      * @brief Baud rate for serial communication.
1012  */
     #define SERIELLE_BAUDRATE 115200
1014
     /**
1016  * @brief Sampling interval in milliseconds.
      */
1018 #define ABTASTINTERVALL_MS 200

1020 /**
      * @brief Initializes hardware components.
1022  *
      * @details
1024  * Configures the light barrier input with internal pull-up,
      * sets the LED as output, and starts serial communication.
1026  */
     void setup()
1028 {
         pinMode(LICHTSCHRANKE_PIN, INPUT_PULLUP);
1030     pinMode(LED_PIN, OUTPUT);

1032     Serial.begin(SERIELLE_BAUDRATE);
         while (!Serial)
1034     {
         }
1036     Serial.print(F("Signal pin:        "));
         Serial.println(LICHTSCHRANKE_PIN);
1038     Serial.print(F("LED pin:           "));
         Serial.println(LED_PIN);
1040     Serial.print(F("Sampling interval: "));
         Serial.print(ABTASTINTERVALL_MS);
1042     Serial.println(F(" ms"));
         Serial.println();
1044     Serial.println(F("Starting measurement..."));
         Serial.println();
1046 }

1048 /**
      * @brief Reads the current state of the light barrier.
1050  *
```

```
 * @return true  if beam is interrupted (LOW)
 * @return false if beam is clear (HIGH)
 */
bool lichtschrankeAusgeloest()
{
    return (digitalRead(LICHTSCHRANKE_PIN) == LOW);
}

/**
 * @brief Controls the built-in LED.
 *
 * @param aktiv true = LED ON, false = LED OFF
 */
void setLED(bool aktiv)
{
    digitalWrite(LED_PIN, aktiv ? HIGH : LOW);
}

/**
 * @brief Outputs the sensor state to the serial monitor.
 *
 * @param ausgeloest       Current sensor state
 * @param zustandGeaendert True if state has changed
 */
void zustandAusgeben(bool ausgeloest, bool zustandGeaendert)
{
    if (zustandGeaendert)
    {
        unsigned long zeitstempel = millis();

        Serial.print(F("["));
        Serial.print(zeitstempel);
        Serial.print(F(" ms] "));

        if (ausgeloest)
        {
            Serial.println("Triggered");
        }
        else
        {
            Serial.println("not triggered");
        }
    }
}

/**
 * @brief Main loop for cyclic sensor polling.
 *
 * @details
 * Reads sensor state, updates LED, and prints changes.
 */
void loop()
{
    static bool letzterZustand = false;
    static bool ersterDurchlauf = true;

    bool aktuellerZustand = lichtschrankeAusgeloest();
    bool geaendert = (aktuellerZustand != letzterZustand) ||
    ersterDurchlauf;

    setLED(aktuellerZustand);
    zustandAusgeben(aktuellerZustand, geaendert);

    letzterZustand = aktuellerZustand;
    ersterDurchlauf = false;

    delay(ABTASTINTERVALL_MS);
}
```

../../Code/TestLichtschranke/TestLichtschranke.ino

4.6.2. Test all Functions

4.7. Further Readings

5. Actor XY

Introduction

WS:cite books, applications, board

5.1. General

General description
cite books

5.2. Specific Sensor/Actor

cite board

5.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

5.4. Library

5.4.1. Description

5.4.2. Installation

- data types
- data structure
- data quantity
- data quality

5.4.3. Functions

5.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

5.5.1. Manual

5.5.2. Inside the Example

5.5.3. Code

5.5.4. Files

5.6. Calibration

cite method

5.7. Simple Code

5.8. Simple Application

5.9. Tests

5.9.1. Simple Function Test

5.9.2. Test all Functions

5.10. Further Readings

Part III.

Domain Knowledge - Tools

6. Python Tool XY

7. Hardware Tool XY

Part IV.

Developmenmt

8. Flow Chart Development XY

Part V.

Monitoring

9. Monitoring XY

Part VI.

Verification - Evaluation - Conclusion

10. Evaluation/Verification XY

11. To DoX Y

12. Conclusion XY

Part VII.

Appendix

13. Bill of Material

14. SBOM

requirements.txt

15. Package Example

15.1. Introduction

15.2. Description

15.3. Installation

```
pip packageExample
```

15.4. Example - Manual

15.5. Example

15.6. Example - Code

15.7. Example - Files

`PackageExample.py`

15.8. Further Reading

References

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

Part VIII.

Hints - Examples for LaTeX - Read, Use and Remove

16. Hints

Please, use **biber.exe**.

If you want to create a symbol directory, call makeindex:
makeindex %.nlo -s nomencl.ist

16.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier curves are formed with the help of Bernstein polynomials. If at least two points and two tangents are known, control points can be determined with which a control polygon is formed. The course of the curve is oriented to this control polygon. The number of control points depends on the degree of the Bézier curve. A Bézier curve of degree n has n+1 control points. The Bézier curve is calculated using the De Casteljau algorithm.

Definition. In the interval [0; 1] the **Bernstein polynomial of n$^{\text{th}}$ degree** is defined by:

$$b_{i,n}(\lambda) = \binom{n}{i}(1-\lambda)^{n-i}\lambda^i, \quad \lambda \in [0,1], \quad i = 0, \ldots, n \tag{16.1}$$

Definition. The binomial coefficient is defined by:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \ldots, n \tag{16.2}$$

Here the Bernstein polynomials $b_{i,n}$ form a basis of the vector space $\mathbb{P}^n(I)$ of polynomials of degree at most n over I. Thus every polynomial of degree at most n can be written uniquely as a linear combination. [Far02]

Beispiel. Determination of Bernstein polynomials for $n = 3$ holds:

$$\begin{aligned}
b_{3,0}(\lambda) &= \binom{3}{0}(1-\lambda)^{3-0}\lambda^0 &&= (1-\lambda)^3 \\
b_{3,1}(\lambda) &= \binom{3}{1}(1-\lambda)^{3-1}\lambda^1 &&= 3\lambda(1-\lambda)^2 \\
b_{3,2}(\lambda) &= \binom{3}{2}(1-\lambda)^{3-2}\lambda^2 &&= 3\lambda^2(1-\lambda) \\
b_{3,3}(\lambda) &= \binom{3}{3}(1-\lambda)^{3-3}\lambda^3 &&= \lambda^3
\end{aligned}$$

$b_{3,i}(\lambda)$ with $i = 0, 1, 2, 3$ are the cubic Bernstein polynomials of degree 3.

Definition. Given are the points

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \ldots, n$$

A **Bézier curve** is then defined by

$$C(\lambda) = \sum_{i=0}^{n} Q_i \cdot b_{i,n}(\lambda) \tag{16.3}$$

The points $Q_i, i = 0, \ldots, n$ are called **control points**.

The control points of a Bézier curve form the so-called control polygon.

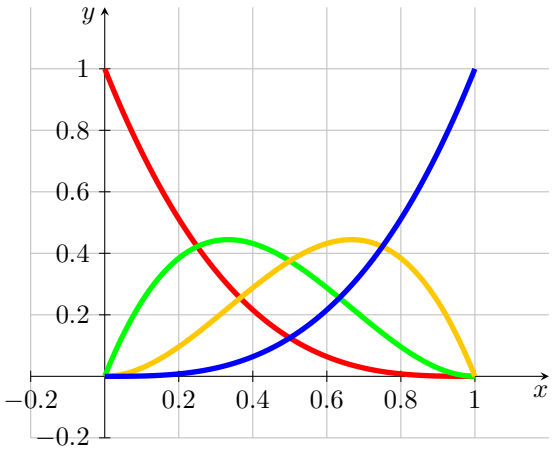


Figure 16.1.: Bernstein polynomial of degree 3

Bemerkung. Let the starting point P_0 and the end point P_1, as well as the tangents $\vec{t}_0$ and $\vec{t}_1$ be given. The tangents are not necessarily normalised.
The control points Q_0, Q_1, Q_2 and Q_3 of the associated Bézier curve are given by the following equations:

$$Q_0 = P_0, \qquad Q_1 = P_0 + \lambda_0 \vec{t}_0, \qquad Q_2 = P_1 - \lambda_1 \vec{t}_n, \qquad Q_3 = P_1 \tag{16.4}$$

[Jak+10]

Beispiel. Given are

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Then, according to theorem **??** for control points of the associated Bézier curve results:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \qquad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \qquad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

The Bézier curve and its control points are shown in the figure **??**.

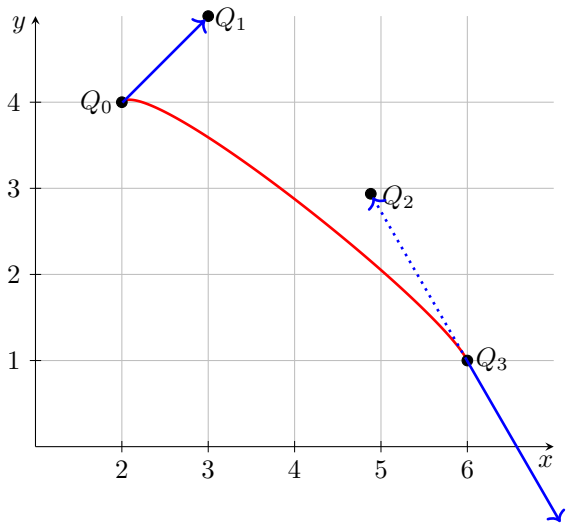


Figure 16.2.: Bézier curve for example 16.1

17. Verrundung mit einem Kreisbogen

17.1. Gleichungen

Blending with an arc is a simple variant of smoothing corners. This variant enables the processing and the generation of files according to DIN 66025. For smoothing, three points P_0 and S and P_1 are always considered, thus a symmetric Hermite problem results. According to theorem **??** it is assumed to be in the standard form $HP(L, \alpha)$.

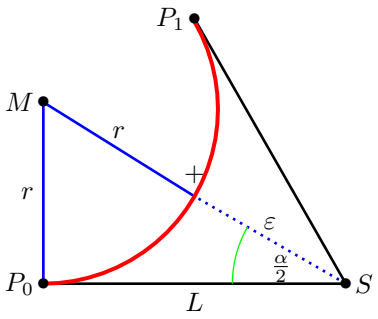


Figure 17.1.: Smoothing a corner with the help of an arc - triangle

The figure **??** represents the situation. The points P_0, S and P_1 are given. The included angle $\alpha = \angle(P_0; S; P_1)$ as well as the distance $L = |||S - P_0||| = |||P_1 - S|||$ are shown in the graph. The red arc is the desired result. The distance of its centre M to S is then $r + \varepsilon$, where r is the radius of the arc and ε is the given tolerance. Thus we obtain a right triangle P_0SM whose edge lengths are L, $r + \varepsilon$ and r.

According to the definition of the sine and the tangent, it follows:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r+\varepsilon} \qquad \text{und} \qquad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \quad \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \qquad \text{und} \qquad r = \cot\left(\frac{\alpha}{2}\right) L$$

The two equations can be combined to give the following conditions:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \qquad \text{bzw.} \qquad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

This gives the equation for the radius:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1-\cos(\alpha)}{1+\cos(\alpha}} = L \cdot \frac{\sin(\alpha)}{1+\cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

The factor for converting L and ε can be simplified using trigonometric transformations.

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1-\sqrt{\frac{1-\cos(\alpha)}{2}}}{\sqrt{\frac{1+\cos(\alpha)}{2}}} = \frac{\sqrt{2}-\sqrt{1-\cos(\alpha)}}{\sqrt{1+\cos(\alpha)}} = \frac{\sqrt{1+\cos(\alpha)}-\sin(\alpha)}{1+\cos(\alpha)}$$

By comparing this with the symmetric Hermite problem in standard form, the following notation is then obtained:

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}}$$

The previous considerations are now summarised in the following sentence.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ uad $\alpha \in (-\pi;\, \pi]$.

Then an arc can be found that connects the points P_0 and P_1, has the same tangent directions at the two points.

For the circular arcs applies:

$$r = L\cdot\left|\frac{P_{1,x}}{P_{1,y}}\right|; \quad \phi_0 = -\operatorname{sign}(\alpha)\cdot\frac{\pi}{2}; \quad \phi_1-\phi_0 = \operatorname{sign}(\alpha)\cdot\pi-\alpha = \beta;$$

$$M = P_0 + \operatorname{sign}(\alpha)\cdot r\cdot\vec{t}_0^{\perp} = \operatorname{sign}(\alpha)\cdot r\cdot\begin{pmatrix}0\\1\end{pmatrix}$$

From the specification of the distance L, the maximum error can be calculated, as shown in theorem **??**. According to the derivation, it is also possible to specify the maximum error ε and determine the maximum distance L from it.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ and $\alpha \in (-\pi;\, \pi]$.

a) Let the maximum error ε be given. The maximum distance L for which an arc exists according to the theorem **??** that takes the error into account is given by:

$$L(\varepsilon,\alpha) = \varepsilon\cdot\frac{P_{1,x}}{\sqrt{P_{1,x}}-P_{1,y}} = \varepsilon\cdot\frac{L+L\cdot\cos(\alpha)}{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}$$

b) When the distance L is specified, the following maximum error results:

$$\varepsilon(L,\alpha) = L\cdot\frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}} = L\cdot\frac{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}{L+L\cdot\cos(\alpha)}$$

These considerations result in limitations for the use of this strategy, which are summarised in the following comment.

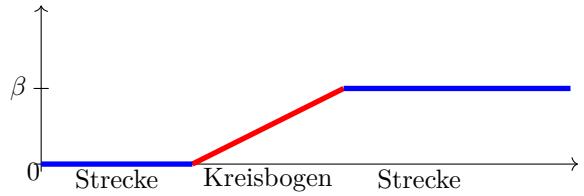


Figure 17.2.: Angular change with blending arc

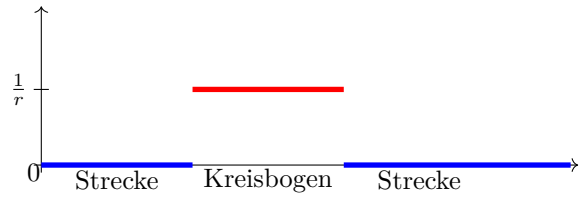


Figure 17.3.: Curvature progression for a blending arc

Bemerkung. The following conditions for applying the strategy of a circle must be fulfilled:

a)
$$P_0 \neq P_1$$

b)
$$\vec{t_0} = -\vec{t_1} \quad \Leftrightarrow \quad \alpha = 0$$

c)
$$\vec{t_0} = \vec{t_1} \quad \Leftrightarrow \quad \alpha = \pi$$

17.2. Bewertung

Rounding by means of a circular arc leads to a continuous course of the angle change. The figure **??** illustrates this.

Due to the rounding with a circular arc, the curvature of the curve is not continuous. The figure **??** shows that the curvature is constant in the range of distances 0 and on the circular arc with the value $\frac{1}{r}$, where r is the radius of the circular arc used. Due to the formula $a = \frac{v^2}{r}$ for a movement on a circular arc, the acceleration course exhibits continuity jumps at the transitions for a constant path velocity.

Depending on the angle change β at the corner S and the tolerance ε, the maximum length of the shortening of the lines can be calculated. The figure **??** shows the corresponding diagram, where the tolerance ε is set to the value 1.

The area provided can also be specified. Depending on the distance L from the corner point S, the deviation ε can be calculated. The figure **??** represents the function as a function of L and the angle α.
The figures **??** and **??** show the curves of the length as a function of the angle change for a fixed tolerance and the error as a function of the angle change for a given length. If the angle change is minimal, then the error or length L is extreme. In this case,

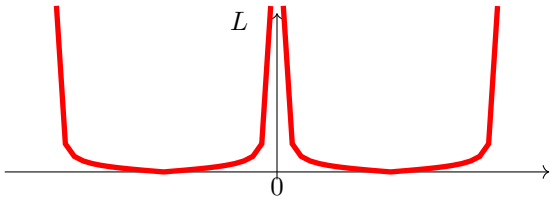


Figure 17.4.: Maximum range for a blending arc of a circle - $L(\varepsilon = const, \alpha)$

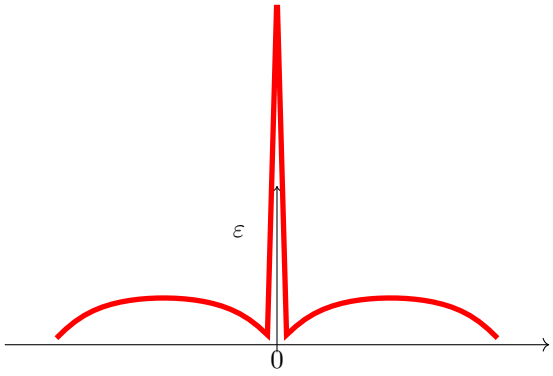


Figure 17.5.: Deviation when specifying the distance L when rounding with a circular arc

rounding by means of an arc is not possible. In order to develop a sensible strategy, a minimum angle change must be specified from which a blemnding arc is permitted. Likewise, a maximum angle change must also be defined. For an angular change of $\pm\pi$ is a bend. In this case, a blending is also not possible.

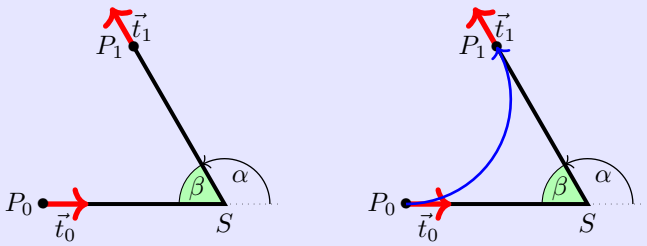
17.3. Examples

Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$

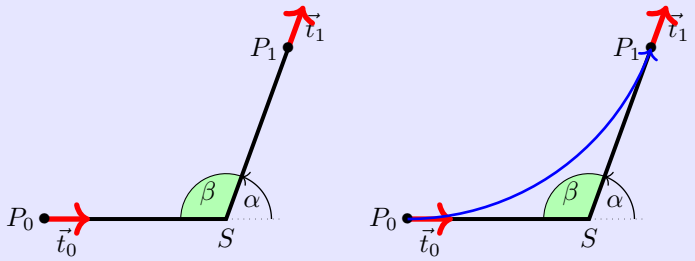


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$

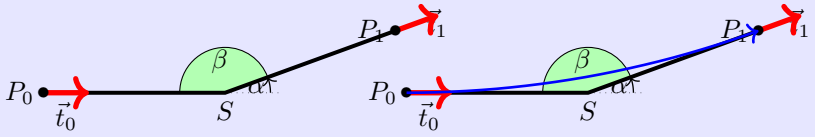


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{1}{9}\pi$.
For the blending arc follows:

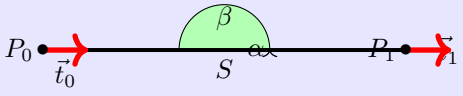
$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$



Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = 0$.
There is no blending arc here.

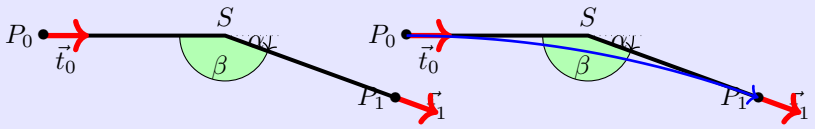


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{1}{9}\pi\right)\\6.0\cdot\sin\left(-\frac{1}{9}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{1}{9}\pi\right)\\\sin\left(-\frac{1}{9}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{1}{9}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-34,0277\end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

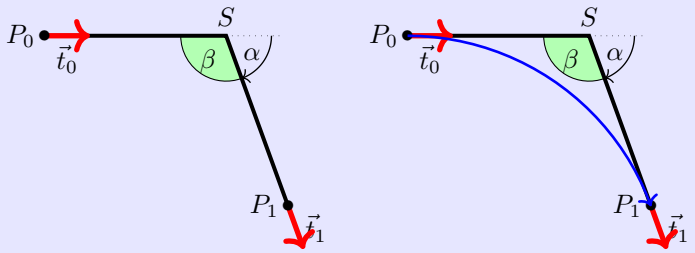


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{7}{18}\pi\right)\\6.0\cdot\sin\left(-\frac{7}{18}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{7}{18}\pi\right)\\\sin\left(-\frac{7}{18}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-8,5689\end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$

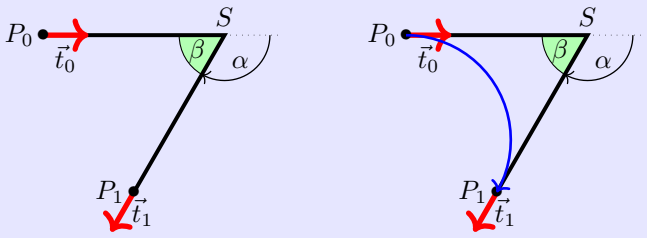


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{2}{3}\pi\right)\\6.0\cdot\sin\left(-\frac{2}{3}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{2}{3}\pi\right)\\\sin\left(-\frac{2}{3}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-3,4641\end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



18. Maple files

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

18.1. Geometries with Maple

The algorithms presented can be well represented using geometries. Two aspects can be investigated. On the one hand, the effort for an implementation, the computational accuracy and the stability of an algorithm are interesting; on the other hand, the methods are to be evaluated and compared with regard to their usefulness. For this purpose, modules for the formula manipulation system Maple [Wat17c] were developed. Maple offers the environment to implement algorithms easily and quickly. Furthermore, graphical representation is possible with simple means. However, a simple workbook of Maple is not designed for very large software projects. Here, one must resort to the possibility of creating and using libraries. On the one hand, Maple offers the possibility of creating one's own libraries, so-called modules. In this way, one remains within the environment and syntax of Maple. This path will be pursued further here. Another possibility is the use of libraries created by means of a higher programming language, e.g. C++, the so-called DLLs.
In the following, the creation and use of a test environment with the help of modules is described. First, the geometry elements that are stored in various modules and their use are described. The structure and use of a module in Maple is then explained so that own extensions and additions are possible.

18.2. Geometry elements used

This is a test environment for the algorithms presented, only geometries that have been described are also used.

- points (`MPoint`)
- lines (`MLine`)
- arcs (`MArc`)
- Bézier curves (`Bezier`)
- polygon courses (`MPolygon`)
- Geometry List (`MGeoList`)
- Hermite problems (`MHermiteProblem`)
- Symmetric Hermite Problems (`MHermiteProblemSym`)

For each geometry element a corresponding module has been created, the name of which is given in the list above. The list of which functions are available is presented in another section.
When implementing such a project, it quickly becomes clear that when using several geometry elements, a clear data structure and well-defined access to the data is essential. For example, when representing straight lines, the decision must be made

whether the representation should be point-directional or by means of two points. The choice is generally made depending on the application. Here, the path is taken that, due to a well-defined access to the data, the use of both representations is possible.

18.3. Building the data structure

The idea is to use a data structure that is as uniform and simple as possible. Maple does offer the possibility to define objects. However, it is difficult to use within a procedural environment. Therefore, all data is basically represented as lists. The first list element always contains an identifier of the element **??**. This is followed by the data, which in turn can be geometry elements. It should be noted that direct access to the data cannot be prevented; here the user is responsible.

Access to the data should be exclusively via procedures of a module. The following is an example of the data structure for points:

```
[MVPOINT,[x,y]]
```

The creation of a point then takes place via a procedure `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

When called up in the test file, the following is then output:

```
TestP0 := ["Point",[10,15]]
```

These data structures are used for the calculation of geometries. They are used in functions or procedures. Unlike variables, the data structures and procedures are defined globally and not locally. Under the command `export` the procedures are declared at the beginning of the module and can thus also be used outside the module. If, for example, a data structure from `MPoint` is to be used in another module or outside the modules, it is called as follows:

```
P0 := MPoint:-New(x,y);
```

In addition to the modules for the geometries, there is a module (MConstant) for saving constants and names. There, for `MVPOINT`, for example. „Point “ or e.g. a constant for the comparison to zero for real numbers.
Finally there is the test file, which is not a module, in which the modules are loaded, called and tested in their function. More about this file later in „1.4 Maple test file “.

18.4. Structure of a module

The following section deals with the purpose of modules and their structure.

The project is about capturing the above geometries in a data structure and representing them in Maple. However, the calculations and formulas that have to be used are a hindrance to the final representation. Modules are very well suited for summarising and hiding the calculations that take place in functions. This way, the important functions can be accessed in Maple without seeing what is in them.

Example:
The function `Angle` from the module `MPoint`: Function to calculate an angle between location vector and x-axis:

```
Angle := proc(P)
   local alpha, x, y;
```

```
   x := GetX(P);
   y := GetY(P);
   alpha := 0;
   if abs(x) < 0.00001
      then
         alpha := 3*Pi/2;
      else
         alpha := Pi/2;
      end if;
   else
      alpha := arctan(y/x);
      if x < 0
      then
         alpha := alpha + Pi;
      else
         if y < 0
         then
            alpha := alpha + 2*Pi;
         end if;
      end if;
   end if;
   return factor(alpha);
end proc;
```

This function is long, but it only represents a small part of the programme for the representation in question. That is why it is in the module and can thus be called externally with a single command:

```
Winkel := MPoint:-Angle(P)
```

The following section describes the structure of a module. Since Maple offers a wide range of possibilities, a restriction is made. Only the structure of the modules used is described, here on the basis of the module`MPoint`. For more detailed possibilities, please refer to the Maple manual [Wat17c].

structure:

The module must first be started. This is done with the name (here always a capital M and the geometry) of the module and the following command:

```
MPoint := module()
```

Then, similar to procedures, the variables and functions must be defined. You can declare them either locally or globally. If the variables or procedures are only used and changed in the module, the declaration is made with the command `local` as follows:

```
 local  Variable names, with, comma, separated;
```

If the variables or procedures are also to be usable outside the module, this is defined with `export`:

```
export Function names, with, comma, separated;
```

This is followed by the specification of the options:
`option package;`,
the description of the module:

```
description “Self-selected module description, e.g. module for points
“;
```

and the initialisation of the module:

```
ModuleLoad := proc
  MVPOINT:=MConstant:-GetPoint();
  print("Module MPoint is loaded");
end proc;
```

From here on, programming is done as usual in Maple. The previously defined procedure and variable names are used and programming is done with commands that are also used outside of a module in Maple. It is important to know that with modules, each function can be called constantly, so the order of the functions is irrelevant.

To finally end the module, use the following command:

```
end module;
```

18.4.1. General structure of a module

In summary, the modules have the following structure:

```
Modulname := module()

   local Names, with, comma, separated;

   export Names, with, comma, separated;

   global Names, with, comma, separated;[1]

   option package;

   description „ Self-selected module description“;


   ModuleLoad := proc()[2]
      MVPOINT:=MConstant:-GetPoint();
      print("Modul MPoint is loaded");
   end proc;

 Procedure1 := proc (Passing parameters)
      inhalt;
   end proc;

   Procedure2 := proc (Passing parameters)
      content;
   end proc;


   ...

end module;
```

[1] Possible, but not used in the modules
[2] Example `MPoint`

18.4.2. Saving a module

The management of modules is done automatically by Maple. However, since the modules are passed on and have to be edited individually, some settings have to be made. Therefore, the following prefix is used for each Maple file of the project:

```
1 restart;
2 with(LibraryTools);
3 lib := "C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

The first line initialises the system. The next 3 lines enable working with archives. In the second line, the tools are loaded so that the variable `lib` can be occupied. All modules are stored in an archive; in this example it is the file `Blending.mla` in the directory `C:/FH/Tools/Maple/MyLibs/`. The command `ThisModule := 'MArc';` contains the name of the current module.

A module is then saved using the command `savelib('ModuleName')`. A file `ModuleName.mla` is then created. Depending on the configuration of Maple, the set path where the file is automatically created may not be writable. Then you can configure the system so that the mla file is stored in the current directory or in a directory of your choice.

If the above prefix is used for a Maple file, all that is required to save the module is `savelib(ThisModule, lib);`

18.4.3. Verwendung eines Moduls

A module that has been saved can now be used in other Maple worksheets. To do this, the command
`with(ModuleName)`
is used. If you have used a special path, Maple cannot find the file `ModuleName.mla`. Then the path must be made known, e.g.:
`savelibname := "c:/Maple/MyLibs";", savelibname;`
Now the procedures of the module can be accessed. An overview of all exported procedures is provided by the command
`Describe(ModuleName)`
is displayed. A list of the names including the names of the transfer parameters is displayed. If a procedure is equipped with the field `description`, this text is also displayed.

18.4.4. Creating a Maple module

Creating your own module is done quickly if you use the framework above. However, one should make some considerations beforehand and maintain a standard.

Before creating an own module, the data model and the corresponding procedures should be worked out. A programme flow chart is useful here. The central task of the module is usually quickly determined. In addition, however, the following rules should be observed.

Rule 1 Basically, both the module and each procedure receive a description. This is not limited to the optional argument `description`, but is placed in front of each procedure. The description basically contains the description of the task. The prerequisite is also mentioned or an error handling is described. Then follows the description of all input parameters, their function and their data structure. The return value is then described.

rule 2 Each module receives a procedure `Version()`, which returns the current version number.

rule 3 Data is not global. To access data, procedures `Set*` and `Get*` are provided. These procedures are also used within the procedures of a module.

rule 4 At least one test function is written for each procedure. The test function can be used to illustrate the use and any special features.

18.5. Functions of the modules

In the following, the individual modules are listed. The data structure of each module and all functions with associated tasks are listed.

18.5.1. Module `MPoint`

`MPoint` is a module for points and works with a data structure and with procedures/functions. The data structure `New` for a point is a list, which is structured as follows:

```
[MVPOINT, [x,y]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Point" and the elements are the x and y coordinates for a point. No distinction is made here between point and vector. A point can also be seen as a vector from the coordinate origin to the point.
Via the Get functions `GetX` and `GetY` one can capture the coordinates of the point and use them in other functions. The other functions can then be used to calculate with the points/vectors.

MPoint		
E	**New**	Data structure for a point
E	**GetX**	Reading the x-coordinate
E	**GetY**	reading the y-coordinate
E	**Angle**	calculating the angle between the x-axis and the point
E	**Add**	Calculates a linear combination of two points/vectors
E	**Sub**	Calculates the difference of two points/vectors
E	**Cos**	Calculates the cosine between two vectors
E	**Sin**	Calculates the sine between two vectors
E	**Scale**	Scales a vector with a factor
E	**Perp**	Calculates a vector that is orthogonal to the given vector
L	**IllustrateXY**	Plot function to illustrate a blue point
E	**Illustrate**	Plot of a blue point
E	**Plot**	Plot of a green point
E	**Plot2D**	Plot of a point with own options
E	**Length**	Calculates the distance of the point to the coordinate origin
E	**Uniform**	Normalises the vector to length 1
E	**LinetoVector**	Calculates vector from a distance
E	**Distance**	Calculates the distance between two points

18.5.2. Module MLine

MLine is a module for lines and works with a data structure and with procedures/functions. The data structure New for a line is a list which is structured as follows:

```
[MVLINE, [P0,P1]]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „Line“ and the elements are the start and end points for a line.
The data structure NewPointVerctor is also a data structure for a line, but here the line is defined by a start point and a direction vector.
The Get functions StartPoint and EndPoint can be used to capture the start and end points of the route and use them in other functions. The other functions can then be used to calculate with the routes.

MLine

E	**New**	Data structure for a line (two-point form)
E	**NewPointVector**	creation of a route (point-direction form)
E	**StartPoint**	reading the starting point
E	**EndPoint**	reading the end point
E	**Position**	Calculate a point that lies on the line
E	**Plot2D**	Plot a part of the route starting from the starting point
E	**Plot2DTangent**	Plot of a point with tangent
E	**Plot2DTangentArrow**	Plot of a point with tangent (as arrow)
E	**LineLine**	Calculation of the intersection of two straight lines.
E	**AngleLine**	Calculation of the angle between two straight lines

18.5.3. Module MArc

MArc is a module for circular arcs and works with a data structure and with procedures/functions. The data structure New for an arc is a list that is structured as follows:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Arc" and the elements are the x- and y-coordinate for the centre, the radius, the start angle and the angle change for an arc.
The Get functions GetM, GetMX, GetMY, GetR, GetPhi, and GetAlpha can be used to collect the data for an arc and use it in other functions. The other functions can then be used to calculate with the arcs.

MArc

E	**New**	Data structure for an arc
E	**GetMX**	Reading the x-coordinate of the centre point.
E	**GetMY**	read the y-coordinate of the centre point
E	**GetR**	read the radius
E	**GetPhi**	reading the start angle
E	**GetAlpha**	Reading the change of angle
E	**GetM**	reading the centre point
E	**Position**	Calculating a point on the arc
E	**Plot2D**	Plot a part of the arc starting from the start angle
E	**Blend**	calculating an arc from a symmetric Hermite problem

18.5.4. module MBezier

The New data structure for a polygon is a list constructed as follows:

```
[MVBEZIER, [PointList]]
```

Within the module MBezier exist the procedures listed in the following table.

MBezier		
E	New	Manual input of control points for a Bézier curve
E	Version	Output of the verions
E	BlendCurvature	Determination of control points from symmetric Hermite problem
E	BlendCurvatureEpsilon	determination of control points from symmetric Hermite problem with given error
E	Position	position on the Bézier curve
E	GetTheta	Reading the angle
E	GetEpsilon	reading the tolerance
E	GetControlPoint	Read the control points
E	Plot2D	Read the Bézier curve
E	PlotControlPoints	representation of all control points

18.5.5. Module MPolygon

MPolygon is a module for polygons and works with a data structure and with procedures/functions. The data structure New for a polygon is a list that is structured as follows:

```
[MVPOLYGON, [PointList]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Polygon" and the elements are any number of points.
Using the Get functions GetPoint and GetN you can get the number of points and the points themselves and use them in other functions. The other functions can then be used to calculate with the points or the polygon course.

MPolygon		
E	New	Data structure for a list of points
E	GetPoint	Reading the ith point from the point list
E	GetN	Determine the number of points in the point list
E	Length	Determination of the Euclidean length of the polygon
E	Position	Calculation of a point on the polygon course
E	Tangents	calculation of the tangent
L	Plot2DAll	Plot list of all points
E	Plot2D	Plot of all points as polygonal plots.
E	Plot2DTangent	representation of the polygon course with tangent
E	PlotPoints	representation of all points

18.5.6. module MGeoList

MeoList is a module for a geometry list and works with a data structure and with procedures/functions. The data structure New for a geometry list is a list that is structured as follows:

```
[MVGEOLIST, []]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „GeoList“ and the elements are any number of individual geometries.
Using the Get functions `GeoGeo` and `GetN`, the i-th element of the list and the number of elements in the list can be captured and used in other functions. The other functions can then be used to calculate with the geometries or the list. In the functions `Length`, `Plot2DAll`, `Plot2D` and `Position` the functions are called in themselves. This is possible because the functions work independently of each other.

MGeoList

E	**New**	Data structure for a geometry list
E	**Append**	Append a geometry element to the data structure
E	**Prepend**	Inserting a geometry element as the first element of the list
E	**Replace**	Replace a geometry element with another one.
E	**GetN**	Determine the number of geometry elements in the list
E	**GeoGeo**	Reading the i-th geometry element
E	**Length**	Calculate the Euclidean length of the geometry elements
E	**Position**	Calculates a point on the geometry
L	**Plot2DAll**	Plot function for plotting the geometry elements
E	**Plot2D**	Plot a part of the geometry list starting from the first element

18.5.7. Module MHermiteProblem

`MHermiteProblem` is a module for Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a route is a list, which is structured as follows:

```
[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „HermiteProb“ and the elements are two points with associated tangents (lines).
Using the Get functions `StartPoint`, `EndPoint`, `StartTangent` and `EndTangent`, the points and their tangents can be captured and used in other functions. The other functions can then be used to calculate with the data for the Hermite problem.

MHermiteProblem

E	**New**	Data structure for a Hermite problem
E	**StartPoint**	reading the start point
E	**EndPoint**	Reading the end point command
E	**StartTangent**	reading the start tangent
E	**EndTangent**	Reading the end tangent
E	**Plot2D**	Plotting the Hermite problem

18.5.8. Module MHermiteProblemSym

`MHermiteProblemSym` is a module for symmetric Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a symmetric Hermite problem is a list built as follows:

```
[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „SymHermiteProb“

and the elements are two points with associated tangents, their intersection, and the distance of the points to the intersection.

Via the Get functions StartPoint, EndPoint, StartTangent, EndTangent, CrossPoint and ParameterL one can acquire the data and use it in other functions. The other functions can then be used to calculate with the data for the symmetric Hermite problem.

MHermiteProblemSym

E	New	Data structure for a symmetric Hermite problem
E	StartPoint	reading the start point
E	EndPoint	Reading the end point command
E	StartTangent	reading the start tangent
E	EndTangent	Reading the end tangent
E	ParameterL	reading of the distance
E	CrossPoint	reading of the intersection point
E	Plot2D	Plotting of the symmetrical Hermite problem
E	Create	creation of a Sym. Hermite problem by 3 points
E	BlendArc	rounding of the corner point by an arc

18.5.9. Module MConstant

MConstant is a module for storing fixed constants and names to identify data structures. In the locally declared functions, starting with CV, the constants for declaring the different geometries are defined. These are names for recognising the geometry elements in the test file. In the globally declared functions, starting with Get, the names for identifying the geometry elements are returned.

MConstant

L	NULLEPS	constant to compare to zero
L	CVPOINT	constant for geometry elements: Point
L	CVLINE	constant for geometry elements: Line
L	CVARC	constant for geometry elements: Arc
L	CVPOLYGON	constant for geometry elements: polygon
L	CVGEOLIST	constant for geometry elements: GeoList
L	CVHERMITEPROBLEM	constant for geometry elements: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Const. for geometry elements: SymHermiteProb
L	CVBIARC	Const. for geometry elements: Biarc
E	GetNullEps	return of the zero comparison command.
E	GetPoint	identifier for points
E	GetLine	Identifier for lines
E	GetArc	identifier for circular arcs
E	GetPolygon	identifier for polygons
E	GetGeoList	Identifier for geometry lists
E	GetHermiteProblemSymmetric	Identifier for symmetric Hermite problems
E	GetHermiteProblem	ID for Hermite problems
E	GetBiarc	identifier for Biarcs

18.5.10. Module MGeneralMath

MGeneralMath is a module for general mathematical functions and works with the data structures and procedures.

MeneralMath

E	MPoint	Data structure for a point
E	MPointX	Reading of the x-coordinate for a point
E	MPointY	reading the y-coordinate for one point
L	MPointIllustrateXY	plot structure for a blue point
E	MPointIllustrate	plot structure for a blue point
E	MPointPlot	Illustration of a green point
E	MLine	Data structure for a line
E	MLineStartPoint	Reading of the start point for a draw frame
E	MLineEndPoint	Reading the end point for a line
E	MPointOnLine	Calculation of a point on the line
E	MLinePlot2D	Plot the part of a line starting at the starting point
. E	MLineLine	calculating the intersection of two linesline

18.5.11. Module Biarc

Data Structure

Requirements and specifications:

The Biarc is to be used to solve a Hermite problem. A Hermite problem is defined as follows:

Given two points P_0 and P_1 with associated normalised tangents $\vec{t_0}$ and $\vec{t_1}$. The biarc must connect the points such that the tangents of the start and end points of the biarc coincide with the tangents of the Hermites problem.

Data structure:

The data structure `New` for a biarc must contain two arcs.

So the data structure for one arc is needed: `MArc:-New`.

This in turn contains the coordinates for the centre, the radius, the start angle and the angle change.

18.5.12. Module MBiarc

`MBiarc` is a module for Biarcs and works with a data structure and with procedures/functions. The data structure `New` for a Biarc is a list which is structured as follows:

```
[MVBIARC, [Arc0, Arc1]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is „Biarc“ and the elements are two arcs.

Using the Get functions `GetArc0` and `GetArc1` one can capture the two arcs for the biarc. The functions listed below can be used to calculate the data for the biarc.

MBiarc		
E	New	Data structure for a biarc
E	GetArc0	Reading of the first arc
E	GetArc1	Reading the second arc
E	Circle	calculating the circle K_J from the Hermite problem data.
E	Plot2DCircle	plot of the circle K_J
E	angle	Calculate the angle from the centre of the circle to points on the circle
E	Plot2D	Plot of the biarc
E	ConnectionPoint	Calculation of the connection point J (Equal Chord)
E	TangentTj	Calculation of the tangent to J
E	Tangent Biarc	rotation of Tj for the biarc.
E	BiarcCenter	Calculate the centres of the arcs of the biarc
E	Biarc	calculate the centre of the biarc.
E	Position	Determine a point on the biarc
E	Blend	Calculate the biarc only from the Hermite problem

18.6. Programme Flowchart

18.6.1. Overall Flow

18.6.2. Verrundung der Kurve

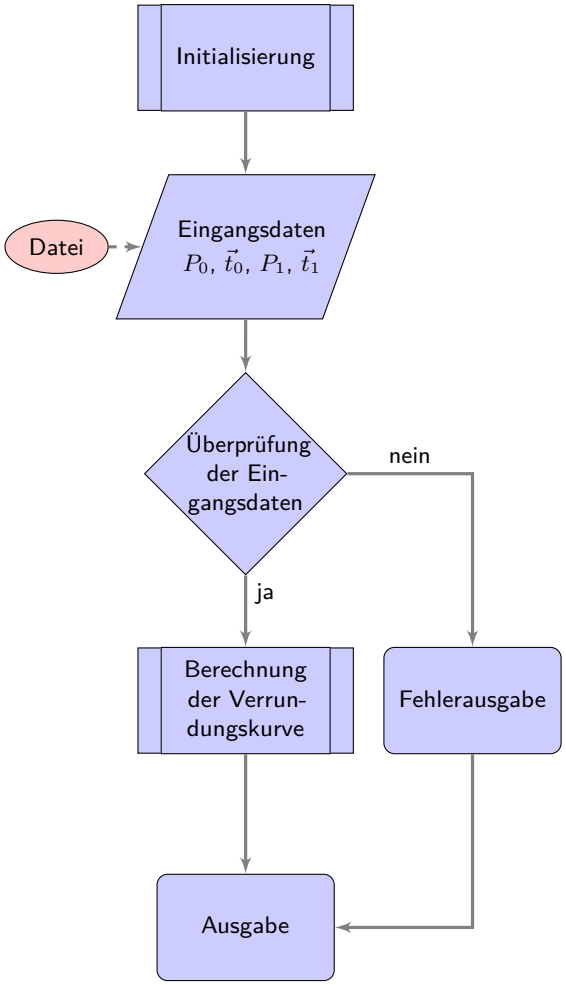


Figure 18.1.: Programme flow chart „Corner rounding“

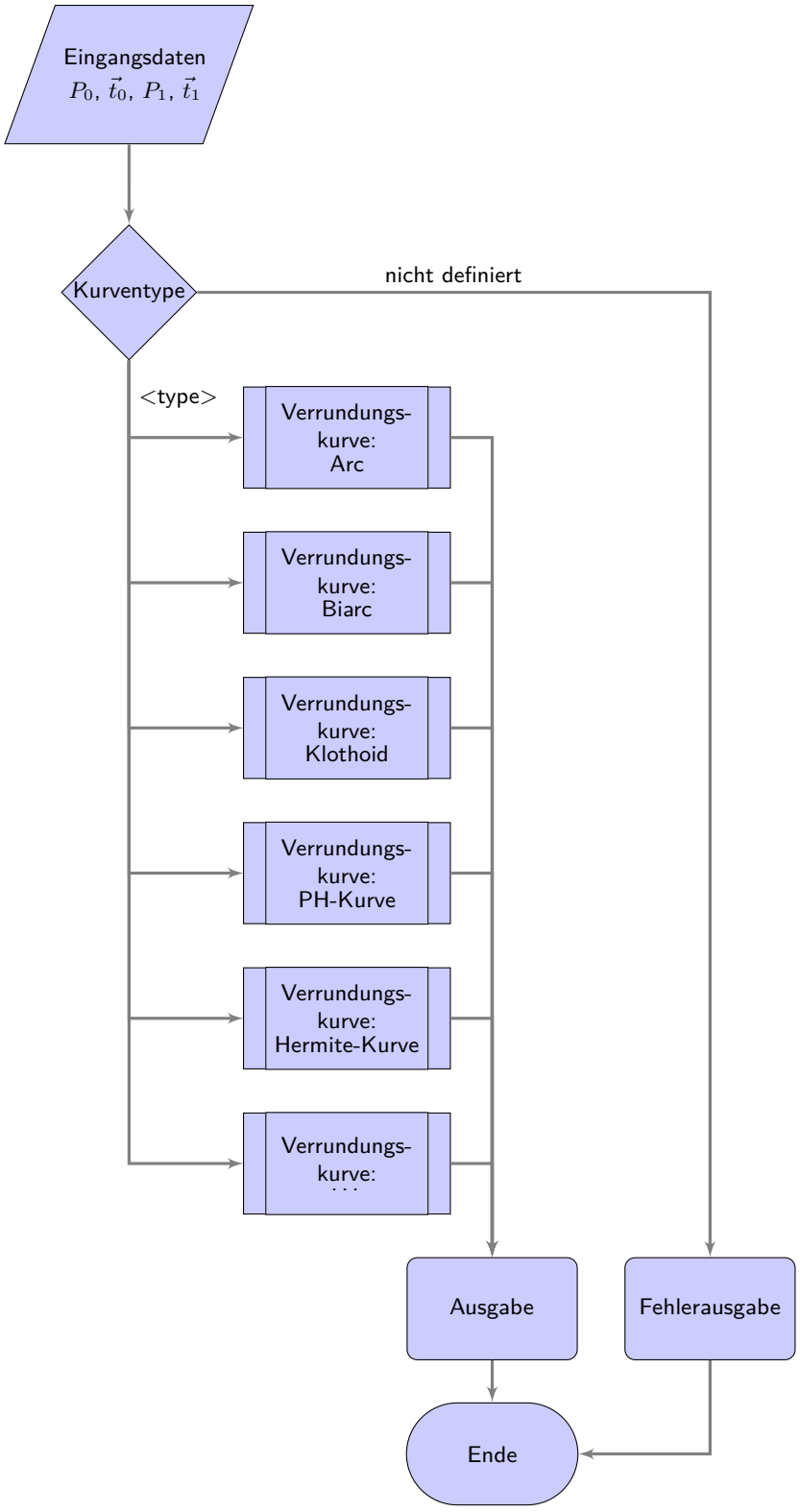


Figure 18.2.: Programme flowchart „Selection of the rounding strategies“

19. Representation of Python Programs

It is possible to integrate a part into the document, see 19.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 19.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```
%%
%% This is file `rename-to-empty-base.tex',
%% generated with the docstrip utility.
%%
%% The original source files were:
%%
%% fileerr.dtx  (with options: `return')
%%
%% This is a generated file.
%%
%% The source is maintained by the LaTeX Project team and bug
%% reports for it can be opened at https://latex-project.org/bugs/
%% (but please observe conditions on bug reports sent to that address!)
%%
%%
%% Copyright (C) 1993-2025
%% The LaTeX Project and any individual authors listed elsewhere
%% in this file.
%%
%% This file was generated from file(s) of the Standard LaTeX `Tools
    Bundle'.
%%
%% ---------------------------------------------------------------------
%%
%% It may be distributed and/or modified under the
%% conditions of the LaTeX Project Public License, either version 1.3c
%% of this license or (at your option) any later version.
%% The latest version of this license is in
%%    https://www.latex-project.org/lppl.txt
%% and version 1.3c or later is part of all distributions of LaTeX
%% version 2005/12/01 or later.
%%
%% This file may only be distributed together with a copy of the LaTeX
%% `Tools Bundle'. You may however distribute the LaTeX `Tools Bundle'
%% without such generated files.
%%
%% The list of all files belonging to the LaTeX `Tools Bundle' is
%% given in the file `manifest.txt'.
%%
 \message{File ignored}
\endinput
%%
%% End of file `rename-to-empty-base.tex'.
```

../../Code/HelloWorld/Blink.py

20. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 20.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 20.1.: The program “Hello World” for an Arduino microcontroller boardis inserted from the file Blink.ino.

```
1000 /**
     *
1002 *  @file Blink.ino
     *
1004 *  @brief Simple program for testing the configuration
     *
1006 *
     *  Turns an LED on for one second, then off for one second, repeatedly.
1008 *
     *  Most Arduinos have an on-board LED you can control. On the UNO, MEGA
         and ZERO
1010 *  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
         set to
     *  the correct LED pin independent of which board is used.
1012 *  If you want to know what pin the on-board LED is connected to on your
         Arduino
     *  model, check the Technical Specs of your board at:
1014 *
     *  https://www.arduino.cc/en/Main/Products
1016 *
     *  modified 8 May 2014
1018 *  by Scott Fitzgerald
     *  modified 2 Sep 2016
1020 *  by Arturo Guadalupi
     *  modified 8 Sep 2016
1022 *  by Colby Newman
     *
1024 *  This example code is in the public domain.
     *
1026 *  https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
     */
1028


1030


1032 /**
     *  @brief the setup function runs once when you press reset or power the
         board
1034 *
     *  standard function of Arduino sketches
1036 *
     *  Initialization of the pin LED_BUILTIN as output
1038 *
     *  @param ---
1040 *
     *  @return void
1042 */
     void setup() {
1044   // initialize digital pin LED_BUILTIN as an output.
       pinMode(LED_BUILTIN, OUTPUT);
1046 }

1048 /**
     *  @brief the loop function runs over and over again forever
1050 *
     *  standard function of Arduino sketches
1052 *
     *  swichting the led on / off for 1sec.
1054 *
     *  @param ---
1056 *
     *  @return void
1058 */

1060 void loop() {
       digitalWrite(LED_BUILTIN, HIGH);    // turn the LED on (HIGH is the
         voltage level)
1062   delay(1000);                        // wait for a second
       digitalWrite(LED_BUILTIN, LOW);     // turn the LED off by making the
         voltage LOW
1064   delay(1000);                        // wait for a second
     }
```

../../Code/Arduino/Blink/Blink.ino

21. First Chapter

...
A Speicherprogrammierbare Steuerung (SPS) is ...

A Computerized Numerical Control (CNC) needs a SPS (PLC) to ...

22. CAGD

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.
The book CAGD by Gerald Farin is a classic on splines. [Far02]

The standard 66025 for programming CNC machines is also a classic; however, it does not deal with splines. [DIN66025-2]

Mr. F. Farouki has dealt with both CNC machine programming and splines. His article[1] on a real-time interpolator also shows this. [FS17]

A new dimension of machine tools have emerged with the invention of 3D printers. [Rus+07]

Another aspect of automation technology is communication. Another milestone has been reached with 5G technology. another milestone has been reached.[2]

[1] co-author is J. Srinathu
[2] Zaf+20.

A. drawings with tikz

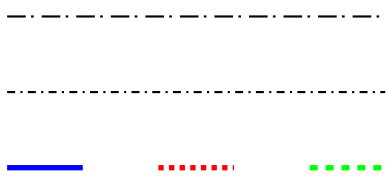
The package tikz is a powerful tool for creating graphics. Many introductions exist. Here only the first steps are shown, so that you can easily create flowcharts.

Drawing a line and arrows

```
\begin{tikzpicture}
    \draw (0,0) -- (1,0);

    \draw[->] (2,0) -- (3,0);

    \draw[<->] (5,0) -- (6,0);
\end{tikzpicture}
```

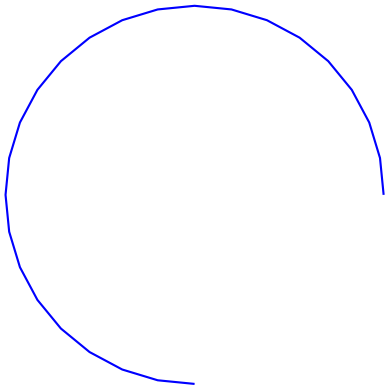

Drawing a thick blue line

```
\begin{tikzpicture}
    \draw[line width=2pt, blue] (0,0) -- (1,0);

    \draw[line width=2pt, red, dotted] (2,0) -- (3,0);

    \draw[line width=2pt, dashed, green] (4,0) -- (5,0);

    \draw [thick,dash dot] (0,1) -- (5,1);
    \draw [thick,dash pattern={on 7pt off 2pt on 1pt off 3pt}] (0,2) -- (5,2);
\end{tikzpicture}
```



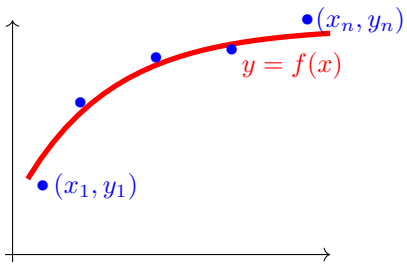
Drawing an arc

```
\begin{tikzpicture}
  \draw [blue,thick,domain=0:270] plot ({5+2.5*cos(\x)}, {1+2.5*sin(\x)});
\end{tikzpicture}
```



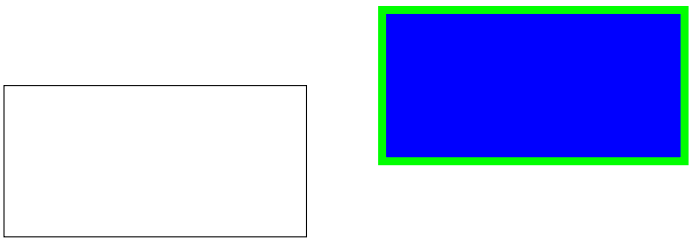
Draw a function

```
\begin{tikzpicture}[
    declare function={%
        F(\x)                   =3-2*pow(2.7979,-0.8*\x);
    }
 ]


    % Zeichnen der Funktion
    \draw[red,line width=2pt,domain=0:4] plot({\x},{F(\x)});

    % Bezeichnung
    \node[red] (O) at(3.5,2.5) {$y=f(x)$};

    % Punkte
    \node[blue] (P1n) at (0.9,0.9) {$(x_1,y_1)$};
    \node[blue] (P1) at (0.2,0.9) {$\bullet$};

   \node[blue] (P2) at (2.7,2.7) {$\bullet$};
   \node[blue] (P3) at (1.7,2.6) {$\bullet$};
   \node[blue] (P4) at (0.7,2) {$\bullet$};

   \node[blue] (P5n) at (4.4,3.1) {$(x_n,y_n)$};
   \node[blue] (P5) at (3.7,3.1) {$\bullet$};

  % Koordinatensystem
   \draw[color=black,->] (-0.2,-0.1) -- (-0.2,3.1);
   \draw[color=black,->] (-0.3,0) -- (4,0);
\end{tikzpicture}
```



Drawing rectangles and moving objects

```
\begin{tikzpicture}
    \draw (0,0) -- (4,0) -- (4,2) -- (0,2) -- cycle;

    \begin{scope}[shift={(5,1)}]
      \draw[green, fill=blue, line width=3pt] (0,0) -- (4,0) -- (4,2) -- (0,2) --
    \end{scope}
\end{tikzpicture}
```

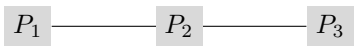


Use of variables

```
\begin{tikzpicture}
\pgfmathsetmacro{\PHI}{-15}
% Now use \PHI anywhere you want -15 to appear,
% can also be used in calculations like 2*\PHI
\def\x{10};

\draw[red] (0,4) -- (1-\PHI*0.5,4);
\draw[green] (0,2) -- (1+1/\x,2);
\draw[blue] (0,0) -- ({1+3*(\x/5+1)},0);
\end{tikzpicture}

\bigskip

%\usetikzlibrary{math} %needed tikz library

\begin{tikzpicture}
  \pgfmathsetmacro{\drehpunktx}{11.63815573}
  %Variables must be declared in a tikzmath environment but
  % can be used outside
  \tikzmath{\x1 = 1; \y1 =1;
  %computations are also possible
  \x2 = \x1 + 1; \y2 =\y1 +3; }
  \draw[->] (\x1, \y1)--(\x2, \y2);
\end{tikzpicture}
```

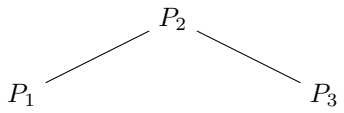
Use of points

```
%\usetikzlibrary{backgrounds}  % is needed

  \begin{tikzpicture}
    \node [fill=gray!30] (P1) at (0,0) { $P_1$ };
    \node [fill=gray!30] (P2) at (2,0) { $P_2$ };
    \node [fill=gray!30] (P3) at (4,0) { $P_3$ };

    \begin{scope}[on background layer]
        \draw (P1) -- (P3);
    \end{scope}
  \end{tikzpicture}
```



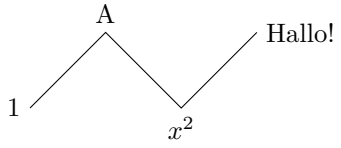
Use of nodes

```
  \begin{tikzpicture}
    \node  (P1) at (0,0){$P_1$};
    \node  (P2) at (2,1){$P_2$};
    \node  (P3) at (4,0){$P_3$};

    \begin{scope}
        \draw (P1) -- (P2) -- (P3);
    \end{scope}
\end{tikzpicture}
```



Use of nodes

```
\begin{tikzpicture}
  \draw (0, 0) node[left]{1}
        -- ++(1, 1) node[above] {A}
        -- ++(1,-1) node[below] {$x^2$}
        -- ++(1, 1) node[right] {Hallo!};
\end{tikzpicture}
```



B. Criteria for a good LaTeX project

A good report is not only characterised by good content, but also fulfils formal aspects. The following list should help to comply with basic rules. Before submitting, all points should be checked and ticked off.

- ☐ Is a suitable directory structure used?
- ☐ Is an appropriate division into files used?
- ☐ Are meaningful directory and file names used?
  - ☐ Are directory and file names such as report or term paper avoided?
  - ☐ Are meaningful names used for images and not „image1“?
  - ☐ Are directory and file names not too long?
  - ☐ Are special characters and spaces avoided?
- ☐ Are commands like \newline and \\ avoided?
- ☐ Are the LaTeXfiles clearly laid out?
  - ☐ Are indentations used in the LaTeXfiles?
  - ☐ Are free lines inserted?
  - ☐ Is the project mentioned in the header of the files?
  - ☐ Does the header of the files mention the main sources?
  - ☐ Is the author mentioned in the header of the files?
- ☐ Are all necessary files given?
- ☐ Are the temporary files deleted?
- ☐ Are graphics created by yourself?
- ☐ Are the sources of the images given?
- ☐ Are citations made with the command \cite?
- ☐ Is a bib file used?
- ☐ Are the entries of the bib-file clearly arranged?
- ☐ Are the entries of the bib-file edited to show them correctly?
- ☐ Are the correct types used for the bib entries?
- ☐ Are meaningful keys used in the bib files?

Unfortunately, the list cannot be complete, but it provides some clues.

Literature

[Aba+16] M. Abadi et al. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[Bar23] J. Bartlett. *Elektronik für Einsteiger*. Springer-Verlag, 2023. ISBN: 978-3-662-66243-4. URL: link.springer.com/book/10.1007/978-3-662-66243-4.

[DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sept. 1988.

[FS17] R. Farouki and J. Srinathu. “A real-time CNC interpolator algorithm for trimming and filling planar offset curves”. In: *Computer-Aided Design* 86 (Jan. 2017). DOI: 10.1016/j.cad.2017.01.001.

[Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.

[Fre15] T. Fredericks. *Bounce2 package*. Accessed Date 06.04.2026. 2015. URL: https://github.com/thomasfredericks/Bounce2.

[HS23] E. Hering and G. Schönfelder. *Sensoren in Wissenschaft und Technik*. 3. [pdf]. Springer Vieweg, 2023. ISBN: 978-3-658-39490-5. DOI: 10.1007/978-3-658-39491-2978-3-662-62697-9.

[Jak+10] G. Jaklic et al. “On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves”. In: *Mathematics of Computation* (2010). [pdf].

[LM12] M. Loeffler-Mang. *Optische Sensorik*. Vieweg+Teubner Verlag, Wiesbaden, 2012. DOI: 978-3-8348-1449-4.

[Lip18] H. H. Lippstadt. *Abstands- und Farberkennungssensor: TCRT5000*. Accessed Date 13.04.2026. 2018. URL: https://wiki.hshl.de/wiki/index.php/Abstands-_und_Farberkennungssensor:_TCRT5000.

[RB26] Roboter-Bausatz.de. *Datenblatt - E18-D80NK Infrarot Hinderniserkennung*. Accessed Date 08.04.2026. 2026. URL: www.roboter-bausatz.de/p/e18-d80nk-infrarot-hinderniserkennung.

[RPF20] M. S. Richard P. Feynman Robert Leighton. *Feynman Lectures on physics, The Millenium eEition*. Basic books, Hachette Book Group, 2020. ISBN: 978-0-465-02493-3.

[Rod23] W. Roddeck. *Einführung in die Mechatronik*. Springer Fachmedien Wiesbaden GmbH, 2023. ISBN: 978-3-658-41949-3. URL: https://link.springer.com/book/10.1007/978-3-658-39491-2.

[Rus+07] D. Russell et al. *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2. 2007.

[Wat17a] Waterloo Maple Inc. 2017. *Description*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description.

[Wat17b] Waterloo Maple Inc. 2017. *Export*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export.

[Wat17c] Waterloo Maple Inc. 2017. *Module*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/maple/view.aspx?path=module.

[Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad.

[Wat17e] Waterloo Maple Inc. 2017. *Option*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option.

[Wat17f] Waterloo Maple Inc. 2017. *Procedures*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure.

[Zaf+20] A. Zafeiropoulos et al. "Benchmarking and Profiling 5G Verticals' Applications: An Industrial IoT Use Case". In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: 10.1109/NetSoft48620.2020.9165393.

[all24] alleloelec. *Eine vollständige Anleitung zum E18-D80NK-einstellbaren IR-Sensor*. Accessed Date 06.04.2026. 2024. URL: https://www.allelcoelec.de/blog/A-Complete-Guide-to-the-E18-D80NK-Adjustable-IR-Sensor.html?utm_source=chatgpt.com.

[tin26] tinkbox. *Proximity Sensor/Switch E18-D80NK- Datasheet*. Accessed Date 08.04.2026. 2026. URL: https://www.rhydolabz.com/e18-d80nk-infrared-obstacle-avoidance-sensor?search=E1.

Index